Informe Técnico: Git Local y Git Remoto

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Git es un sistema de control de versiones **distribuido**. Esa palabra explica la diferencia que este documento desarrolla: en Git no hay una copia privilegiada del proyecto y otras subordinadas, sino copias completas que se sincronizan entre sí. Lo que llamamos «repositorio local» y «repositorio remoto» son dos de esas copias, y lo que las distingue es el papel que cumplen, no su naturaleza.

Confundirlas es la causa del malentendido más común entre quien empieza: creer que `git commit` sube el trabajo a internet. No lo hace. Registra la versión en el equipo, y hasta que no se ejecuta `git push` ese trabajo existe en un solo disco.

2. Objetivo

Establecer las diferencias entre el repositorio Git local y el repositorio Git remoto, describiendo las características de cada uno, y documentar los comandos básicos que operan sobre ambos.

3. Diferencias entre Git local y Git remoto

Característica	Git local	Git remoto
Dónde reside	En el equipo del desarrollador, dentro de la carpeta oculta `.git/`	En un servidor accesible por red: GitHub, GitLab, Bitbucket o un servidor propio
Necesita conexión	No. Registrar versiones, crear ramas y consultar el historial funcionan sin internet	Sí. Toda operación contra él requiere red
Quién accede	Solo quien usa ese equipo	Todo el equipo de trabajo, según los permisos configurados
Contenido	Historial completo, área de preparación y copia de trabajo	Historial completo, sin copia de trabajo ni área de preparación
Función principal	Desarrollar: escribir, probar y registrar versiones	Respaldar, compartir y servir de punto de encuentro del equipo
Velocidad	Inmediata: todo ocurre en disco	Depende de la red y del tamaño de lo transferido
Riesgo si se pierde	Se pierde solo lo que aún no se envió al remoto	Se recupera desde cualquier copia local, que tiene el historial completo
Cómo se crea	`git init` o `git clone`	Desde la interfaz del servicio, o con `gh repo create`
Integración continua	No la ejecuta	Dispara las verificaciones automáticas al recibir cambios

> **La consecuencia práctica.** Un repositorio local sin remoto es trabajo que vive en un solo disco. Es exactamente la situación en la que estuvo este proyecto: trece versiones registradas, con todo su historial, sin ninguna copia fuera del equipo.

4. Las tres áreas del repositorio local

Antes de los comandos conviene nombrar las tres áreas por las que pasa un cambio, porque casi todos los comandos básicos mueven archivos de una a otra:

Área	Qué contiene
Copia de trabajo	Los archivos tal como se editan
Área de preparación (staging)	Lo que se ha marcado para entrar en la próxima versión
Repositorio	Las versiones ya registradas, en `.git/`

`git add` mueve de la copia de trabajo a la preparación; `git commit`, de la preparación al repositorio; `git push`, del repositorio local al remoto.

5. Comandos básicos del repositorio local

Comando	Descripción
`git init`	Crea un repositorio nuevo en la carpeta actual: genera `.git/` y empieza a seguir el proyecto
`git config --global user.name "nombre"`	Define el nombre que firmará las versiones
`git config --global user.email "correo"`	Define el correo asociado a las versiones
`git status`	Muestra qué archivos cambiaron, cuáles están preparados y en qué rama se está
`git add archivo`	Prepara un archivo para la próxima versión
`git add .`	Prepara todos los cambios de la carpeta actual hacia abajo
`git commit -m "mensaje"`	Registra una versión con lo preparado y su mensaje descriptivo
`git log`	Muestra el historial de versiones con autor, fecha y mensaje
`git log --oneline`	El mismo historial, una línea por versión
`git diff`	Muestra las diferencias aún no preparadas
`git diff --staged`	Muestra las diferencias ya preparadas, antes de registrarlas
`git branch`	Lista las ramas; con un nombre, crea una nueva
`git switch rama`	Cambia a otra rama
`git switch -c rama`	Crea una rama y cambia a ella en un solo paso
`git merge rama`	Integra en la rama actual los cambios de otra
`git restore archivo`	Descarta los cambios no preparados de un archivo
`git revert versión`	Crea una versión nueva que deshace otra, sin borrar el historial
`git tag nombre`	Marca una versión con una etiqueta, para señalar entregas

6. Comandos básicos del repositorio remoto

Comando	Descripción
`git remote add nombre url`	Asocia un repositorio remoto al local, bajo un nombre corto (habitualmente `origin`)
`git remote -v`	Lista los remotos configurados con sus URL de lectura y escritura
`git remote remove nombre`	Desvincula un remoto
`git clone url`	Descarga un repositorio remoto completo y configura `origin` automáticamente
`git push remoto rama`	Envía al remoto las versiones registradas localmente
`git push -u origin main`	Envía y además deja la rama local enlazada a la remota, para poder usar `git push` a secas después
`git fetch remoto`	Trae del remoto las versiones nuevas **sin** aplicarlas a la copia de trabajo
`git pull remoto rama`	Trae del remoto y además integra los cambios en la rama actual
`git branch -r`	Lista las ramas que existen en el remoto
`git push remoto --delete rama`	Elimina una rama del remoto
`git push --tags`	Envía al remoto las etiquetas creadas localmente

> **`fetch` frente a `pull`.** Es la distinción que más confusión causa. `fetch` solo descarga: deja el trabajo intacto y permite mirar qué llegó antes de decidir. `pull` descarga e integra, y puede producir conflictos en mitad de algo que se estaba escribiendo. Ante la duda, `fetch` primero.

7. Flujo completo: del equipo al remoto

La secuencia habitual de trabajo, con las tres áreas de por medio:

```
$ git status                    # ¿qué cambió?
$ git add .                     # preparar los cambios
$ git commit -m "mensaje"       # registrar la versión (local)
$ git push origin main          # enviarla al remoto
```

Y la secuencia para empezar a trabajar con un repositorio ya existente:

```
$ git clone https://github.com/usuario/proyecto.git
$ cd proyecto
$ git switch -c mi-rama          # aislar el trabajo nuevo
  ... editar, probar, registrar versiones ...
$ git push -u origin mi-rama     # publicar la rama
```

8. Aplicación en el proyecto MONK GRAIN

El proyecto emplea las dos ramas que ilustra este documento: `main` conserva la última versión verificada, y las correcciones se desarrollan aparte hasta que pasan las comprobaciones automáticas.

```
$ git branch
  correcciones-auditoria
* main

$ git log --oneline -3
e02eed6 El porcentaje de la comisión, con su espacio
360ea1d Auditoría del estado y corrección de sus hallazgos
00db999 Rastros del producto anterior en la interfaz
```

9. Conclusión

El repositorio local y el remoto no son dos herramientas distintas, sino dos copias del mismo proyecto con papeles complementarios: el local es donde se trabaja y se registra la historia sin depender de la red; el remoto es donde esa historia queda respaldada, se comparte con el equipo y dispara las verificaciones automáticas.

Entender que `git commit` no sube nada, y que hasta `git push` el trabajo vive en un solo disco, es lo que evita la pérdida más común y más evitable de un proyecto.